

Network Science Data II

PHYS 7332, Fall 2025

Prof. Brennan Klein

Assignment 2, due by Monday, Oct. 20, 11:00pm ET.

Please submit a .pdf of your answers via Canvas, including a url to your .ipynb notebook in your fork of our class repository. Please name your Canvas submission as Lastname.pdf, (e.g. Klein.pdf). If you need help with LaTeX or GitHub, please visit us during office hours or post questions on our email thread.

1. *Question 1 (15 points). More databases.*

There are lots of databases out there that are very different from SQL. Although SQL is probably what you'll see most often out in the real world, it isn't optimal for all use cases. Let's read up on a few other kinds of databases to better understand what's out there. For each of the databases listed below, please explain how it structures its data (e.g. key/value, tabular, etc.) and tell us what kinds of data/applications it's most useful for. Additionally, give us an example of a bad use case for that specific database and explain why it is poorly suited for that particular use case.

- (a) Redis
- (b) MongoDB
- (c) Cassandra
- (d) Neo4j

2. *Question 2 (40 points). Exploring community detection.*

- (a) Download a network dataset from <https://networkrepository.com/> with between $1,000 < N < 5,000$ nodes (select a different dataset from the one you used in Assignment 1). Select a modularity maximization algorithm to detect communities in this graph.
 - i. Visualize your network, coloring the nodes by community membership.
 - ii. Report the modularity of the partition your algorithm found.
 - iii. Report the number of communities detected.
 - iv. Store your partition as a dictionary, in the form of `{node_id: community_id}`.
- (b) Using the same network as in (a), use `graph-tool` to create a partition of your network (i.e., run community detection). Visualize your network again, coloring the nodes by community membership; use the same network visualization layout as the in (a). Store your partition as a dictionary, in the form of `{node_id: community_id}`.

- (c) Create a degree-preserving randomization of your network, and run the two community detection algorithms from above on your randomized graph. Store each partition as a dictionary, in the form of `{node_id: community_id}`.
- (d) You should now have four different dictionaries, corresponding to the partitions created in (a-c). Create a figure with four subplots: In each, plot the community size distribution of each partition. Your figure should be publication-ready quality, with well-labeled, consistent axes, log-scaled where relevant, etc.
- (e) For the different partitions computed above, discuss how you would approach comparing these partitions quantitatively. For example, imagine a function $f(b_1, b_2)$ that takes two partitions (b_1 and b_2) as input and returns a number corresponding to how similar/dissimilar the partitions are. What properties should this function f have? For this question, base your answer on a technique used in the literature, after a brief search about methods for quantifying differences between graph partitions.

3. Question 3 (30 points). Benchmarks for community detection.

There are many ways to “benchmark” community detection algorithms; one famous approach is known as the Lancichinetti-Fortunato-Radicchi (LFR) benchmark¹.

- (a) Read the LFR Benchmark paper, linked in the footnote below. Describe the core motivation behind the paper, its main contributions, and the basics of the benchmark (i.e., what are the inputs to the benchmark, what is the output, etc.).
- (b) On page 2 of the article—in *Section II: The Benchmark*—the authors describe the five steps involved in sampling benchmark graphs.
 - i. Step 1 is reminiscent of the procedure for creating powerlaw configuration model networks. Write a function that outputs a powerlaw degree sequence, $\{k_1, k_2, \dots, k_n\}$, which can be used as input in a configuration model. Your function should take n (network size), γ (powerlaw degree exponent), and $\langle k \rangle$ (average degree) as inputs.
 - ii. Step 2 introduces a mixing parameter, μ , that is used to assign a fraction of a node’s total degree, k_i , to be *internal* community edges (i.e., connecting to nodes with the same community label as node i), such that $k_i^{in} = k_i(1 - \mu)$; with probability μ , therefore, the remaining edges of i are connected to nodes *external* to the community, $k_i^{out} = k_i\mu = k_i - k_i^{in}$. Write a function that implements this step.
 - iii. Step 3 resembles Step 1, but instead of defining powerlaw degree values, it defines a powerlaw community size distribution. What additional considerations does the paper say are needed in order to implement this step? No code is required to answer this.
 - iv. (Optional): *Finish writing the rest of the function, putting Steps 1-5 together. Note: This is a challenging function to implement and, as such, is optional.*

¹ Lancichinetti, A., Fortunato, S., & Radicchi, F. (2008). Benchmark graphs for testing community detection algorithms. *Physical Review E*, 78(4), 046110. doi: [10.1103/PhysRevE.78.046110](https://doi.org/10.1103/PhysRevE.78.046110).

If you don't have time to implement this, feel free to explore existing (proper) implementations of this algorithm, either online or with AI assistance.

- (c) `networkx` has a built-in function, `nx.LFR_benchmark_graph()`, that can generate these benchmark graphs. Many users of this algorithm report its inconsistency in convergence. Explore this function's usage for yourself: Are there certain parameter combinations that make the algorithm converge more/less successfully? Based on your reading of the original LFR paper, and your experiments above, briefly describe why you think this function has difficulty converging.

4. *Question 4 (15 points). Python packages and software management.*

Sometimes, you want to put a chunk (or several related chunks) of Python code into a package that you or others can install. This helps you and others use your code across several different projects and organize it in ways that make sense for you. You've developed several useful functions in this class so far; now it's time to turn them into a package for your future use. You may find [this tutorial](#) and its [sequel](#), on Python packaging and making a package installable, to be useful.

- (a) Distill some of the code we've written in this class so far into a basic toolkit that you'll reuse. Pick 6-10 functions (or objects) that you think would make up a useful network science toolkit based on code we've worked on in class. Note: You should not be planning on writing much *new* code for this question!
- (b) Create a new GitHub repository with a `README.md` file that thoroughly explains the functions and/or objects you put in this package. Link it in your answer.
- (c) Following the tutorials linked above, copy your code into a file(s) in your new repository, making sure it's organized in a way that works for you.
- (d) Again, following the tutorials linked above, make your repository/package pip installable. If everything works correctly, we should be able to clone your repository, `pip install` it into an environment, and import your functions/objects.

5. *(Optional) Question 5 (5 bonus points).*

For every problem set this semester, there will be an opportunity to earn extra credit. Since this is a new course, we are eager to receive genuine feedback on the material, pace, instruction style, etc. of the course so far. Please provide any feedback about the lectures in recent weeks and/or this assignment.